

PROJECT REPORT

Myntra GENZ Shopping Automation using Selenium Python and BDD Framework

Submitted By

Name: Sujal Gupta

Superset-ID: 5386307

Repository Name: Sujal_5386307_WiproCapstone

1. Introduction

Software testing is an essential phase in the software development life cycle. Manual testing becomes difficult when the number of test cases increases because repeated execution consumes time and can lead to human errors.

E-commerce applications such as Myntra involve multiple workflows including category navigation, product selection, shopping bag validation, and complete purchase flow. Repeatedly validating these functionalities manually is inefficient.

To address this challenge, an automation framework was developed for Myntra GENZ shopping functionalities using Selenium with Python.

Two frameworks were implemented:

1. Selenium + Python + Pytest + Page Object Model (POM)
2. Selenium + Python + Behave (BDD) + Page Object Model

The system automates user actions and generates execution reports, logs, screenshots, and test evidence.

2. Problem Statement

Manual testing of Myntra shopping functionalities is repetitive and time-consuming. Human testers repeatedly validate category navigation, product selection, shopping bag operations, and complete user journeys.

Challenges:

- Time-consuming execution
- Human errors during repetitive testing
- Lack of reusability
- Difficulty maintaining large test suites
- Reduced reporting capability

Automation was implemented to solve these limitations.

3. Objectives

The major objectives of this project are:

- Automate Myntra GENZ shopping functionalities
- Reduce manual effort
- Increase testing speed and accuracy
- Build reusable architecture using POM
- Implement positive and negative testing

Execute complete End-to-End workflow • Capture logs and screenshots • Generate reports automatically • Compare Pytest and BDD implementations

4. Scope of Project

The project automates major GENZ categories of Myntra and validates shopping workflows.

Covered functionality:

- Category navigation • Product page validation • Shopping bag validation • End-to-End order flow • Positive test cases • Negative test cases • Reporting and logs
-

5. Technology Stack

Programming Language: Python Automation Tool: Selenium WebDriver Framework: Pytest, Behave (BDD) Design Pattern: Page Object Model Browser: Google Chrome IDE: Visual Studio Code Version Control: GitHub Reporting: Allure, JSON, CSV, Excel Libraries:

- Selenium • Pytest • Behave • Allure • Pandas • OpenPyXL • WebDriver Manager
-

6. System Architecture

The project follows Page Object Model architecture.

POM separates:

- Test logic • Page locators • Reusable methods

Advantages:

- Easy maintenance • Better reusability • Reduced code duplication
-

7. Selenium + Pytest Project Structure

tests/ pages/ utils/ conftest.py pytest.ini screenshots/ reports/ logs/

Description:

pages/: contains reusable page methods

tests/: contains test execution files

utils/: utility methods and driver setup

reports/: CSV, Excel, JSON reports

screenshots/: failure screenshots

logs/: execution logs

8. BDD Project Structure

features/ genz.feature environment.py

features/steps/ genz_steps.py

pages/ utils/ logs/ screenshots/

Description:

Feature file contains business scenarios.

Step definition files connect feature steps with Selenium methods.

9. Test Strategy

Testing strategy focuses on:

Functional Testing End-to-End Testing Positive Testing Negative Testing BDD Scenario Testing
Reporting Logging

Framework Features:

- Reusable methods
- Automatic screenshots
- Allure dashboard
- Logging mechanism
- Excel report generation

10. Implemented Test Scenarios

GENZ Categories:

Women's Western Wear: Dresses Under ₹599 (End-to-End)

Men's Casual Wear: T-Shirts (Positive)

Men's Occasion Wear: Kurtas (Positive)

Men's Footwear: Casual Shoes (Positive)

Beauty and Grooming: Skincare (Positive)

Accessories: Jewellery (Negative)

Women's Footwear: Heels (Negative)

11. End-to-End Workflow

Complete E2E flow:

Step 1: Open Myntra Website

Step 2: Hover over GENZ menu

Step 3: Open Women's Dresses category

Step 4: Display products

Step 5: Open first product

Step 6: Select size

Step 7: Add to bag

Step 8: Open Shopping Bag

Step 9: Verify item present in bag

Step 10: Click Place Order

Expected Result: Complete shopping flow should execute successfully.

12. Positive Testing

Positive testing validates expected behavior.

Examples:

T-Shirts page opens Kurtas page opens Skincare page opens Casual Shoes page opens

Expected Result: Application behaves correctly.

13. Negative Testing

Negative testing validates invalid conditions.

Jewellery Test: Verifies Smartphone category should not exist.

Heels Test: Verifies Refrigerator category should not exist.

Expected Result: Invalid conditions correctly handled.

14. BDD Implementation

BDD follows Given-When-Then approach.

Example:

Feature: Myntra GENZ Shopping

Scenario: Women's Dresses E2E

Given user opens Myntra When user hovers GENZ And user clicks dresses Then products should be visible

Advantages:

Readable language Easy communication Requirement based testing

15. Reporting

Reports implemented:

1.JSON Report 2.CSV Report 3.Excel Report 4.Allure Report 5.Screenshots 6.Execution Logs

Allure includes:

Pass/Fail summary Execution details Screenshots Attachments Logs

16. Logging

Execution logs capture:

Browser launch Navigation GENZ hover Category selection Product selection Add to bag Shopping bag actions Place order actions Errors

Generated file:

logs/execution.log

17. Data Source

No static dataset is used.

Data source: Myntra website itself.

Selenium dynamically extracts:

Product names Prices Categories Shopping bag items Titles DOM elements

Locators used:

XPath CSS selectors Class names

18. Challenges Faced

GENZ hover issue Dynamic locators Multiple similar elements Window handling Dynamic product pages Allure integration Screenshot attachment BDD integration

19. Results

Automation successfully executed:

Positive scenarios Negative scenarios End-to-End flow BDD scenarios Reporting Screenshots Logs

The framework reduced manual effort and improved efficiency.

20. Conclusion

The developed automation framework successfully automated Myntra GENZ workflows using Selenium with Python.

Both implementations (Pytest and BDD) demonstrated reusable architecture and effective automation practices.

The framework supports reporting, screenshots, logs, and complete shopping flow validation.
